Class 18: ML Modeling

Data-driven Systems Engineering

Roadmap of the class

1. **Frame the modeling problem**
   Hypothesis class, loss, target, error costs, and operational constraints.
2. **Compare model families**
   Linear models, trees, probabilistic models, neural networks, and their assumptions.
3. **Use the scikit-learn interface**
   The estimator contract: `fit`, `predict`, and `transform`.
4. **Build reproducible pipelines**
   Column-wise preprocessing and the `Pipeline` abstraction.
5. **Select through evidence**
   Train/test splits, cross-validation, fair comparisons, and hyperparameter search.
6. **Close the modeling loop**
   A decision-tree case study, references, and the transition to experiment tracking.

ML Modeling

From a modeling decision to a reproducible training pipeline

Problem framing $\rightarrow$ model families $\rightarrow$ scikit-learn $\rightarrow$ evidence

What is modeling?

- A model is a function $f : \mathcal{X} \to \mathcal{Y}$ chosen from a hypothesis class $\mathcal{H}$ (e.g. linear functions, decision trees) so that it generalizes to unseen samples.
- Learning selects f by minimizing the empirical risk on the training data,

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i)),$$

  where the loss L quantifies the cost of an error; the quantity of interest is the expected risk on new data.
- The choice of $\mathcal{H}$ encodes assumptions about structure, noise, and decision boundaries, and determines the trade-off among accuracy, interpretability, computational cost, and maintainability.

Questions that precede the choice of a model



Which **decision or prediction** does the model support?



What is the **target space**: a class, a score, or a continuous value (cf. Class 13)?



Which **errors are costly**, and how costly relative to each other?



What are the **latency constraints** for training and inference?



How frequently is the **data distribution** expected to change?

Question 1 of 5: decision or prediction



Which decision or prediction does the model support?

A. An automatic action

- Example: label each bottle as premium or standard on the price list, with no human review.
- The model must output a **reliable class (and probability)** the system can act on.
- Favors models with reliable scores (logistic regression, gradient boosting) and a documented decision rule.

B. Support for a human

- Example: rank wines for a sommelier to taste first.
- Only the **ordering** of cases matters, and the sommelier needs a reason for each ranking.
- Favors ranking metrics (precision at k) and interpretable outputs.

Question 2 of 5: target space



What is the target space: a class, a score, or a continuous value?

A. A class

- Example: premium or standard wine (quality ≥ 7 or not).
- Use a **classifier**; optionally expose `predict_proba` for a score.
- Evaluate with precision, recall, and the confusion matrix, not with error in units.

B. A continuous value

- Example: the wine quality score itself; pasteurization temperature.
- Use a **regressor**; the loss is measured in the units of the target.
- Evaluate with MAE, RMSE, or R^2; the output is the answer itself.

Question 3 of 5: costly errors



Which errors are costly, and how costly relative to each other?

A. A missed positive is worse

- Example: a premium wine sold as standard loses its margin, which costs more than the opposite error.
- Favor **recall**: use class weights or oversample the rare class.
- Accept more false positives, and budget the returns or re-tastings they create.

B. A false alarm is worse

- Example: a standard wine sold as premium triggers returns and damages the brand.
- Favor **precision**: label premium only on strong evidence.
- Accept that some positives will be missed; make this explicit to the stakeholders.

Question 4 of 5: latency



What are the latency constraints for training and inference?

A. Strict, real-time inference

- Example: a response within milliseconds inside an API.
- Favor **light models**: linear models, shallow trees, a small ensemble.
- Preprocessing must be cheap, and the full `Pipeline` must be loaded once at start-up.

B. Offline, batch scoring

- Example: scoring every bottle of a new vintage in one batch.
- Large ensembles and **extensive hyperparameter search** are affordable.
- The limit is the total training budget, which must be tracked per run.

Question 5 of 5: distribution change



How frequently is the data distribution expected to change?

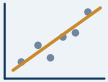
A. Stable distribution

- Example: a controlled laboratory measurement.
- A **fixed model** retrained on a schedule is sufficient.
- Monitor a few input statistics and the error, and register each retrained version.

B. Fast-changing distribution

- Example: a sensor stream with seasonal or process drift.
- Favor **online or incremental models** and drift detection (Class 28).
- Evaluate on the most recent data, not on a random split of the whole history.

Model families in context: supervised learning



Linear models

Efficient, interpretable, strong baselines.

Logistic regression
Bishop (2006), §4.3



Kernel models

Maximum margin; non-linear through kernels.

Support vector machine
Cortes & Vapnik (1995)



Instance-based

Predict from the closest training samples.

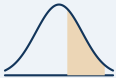
k-nearest neighbours
Cover & Hart (1967)



Trees and ensembles

Tabular data, interactions, boosting.

Random forest
Breiman (2001)



Probabilistic models

Explicit uncertainty and structure.

Naive Bayes
Domingos & Pazzani (1997)



Neural networks

Large unstructured data, highly non-linear.

Multilayer perceptron
Rumelhart et al. (1986)

Model families in context: beyond the supervised toolbox



Clustering

Groups of similar samples, no labels.

k-means
Lloyd (1982)



Dimension reduction

Fewer dimensions that preserve variance.

PCA
Jolliffe (2002)



Anomaly detection

Flags samples unlike the normal data.

Isolation forest
Liu et al. (2008)



Sequence models

Temporal dependence in the data.

Hidden Markov model
Rabiner (1989)



Online learning

Any family, updated incrementally (Class 28).

Hoeffding tree
Domingos & Hulten (2000)

Start simple.
Build a baseline first, then justify each step in complexity.

scikit-learn as an architecture: one interface for every model

Estimator	`fit(X, y)`	Learns from data and stores the result in attributes ending with an underscore (e.g. `coef_`).
Predictor	`predict(X)`	Maps new samples to labels or values; classifiers also expose `predict_proba`, and all predictors expose `score`.
Transformer	`transform(X)`	Rewrites the features (scaling, encoding, imputation); `fit_transform` is the shortcut for the training split.
Meta-estimator	`fit / predict`	Wraps other estimators and exposes the same interface (`Pipeline`, `GridSearchCV`).

The fit / predict contract

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X_train)       # learns mean, s.d.
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)          # same parameters

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train_s, y_train)                # learning happens here

y_pred = model.predict(X_test_s)             # labels (tau = 0.5)
y_prob = model.predict_proba(X_test_s)[:, 1]  # scores
```

Pipeline: preprocessing and model as a single estimator

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
models = {
    "logistic": LogisticRegression(C=1.0, max_iter=1000),
    "svm": SVC(kernel="rbf", C=10, gamma="scale"),
    "forest": RandomForestClassifier(n_estimators=200, max_depth=8),
}
for name, clf in models.items():
    pipeline = Pipeline([("scaler", StandardScaler()),
                         ("model", clf)])
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
```

Experiment tracking and search

How do we compare models fairly and keep a record of every run?

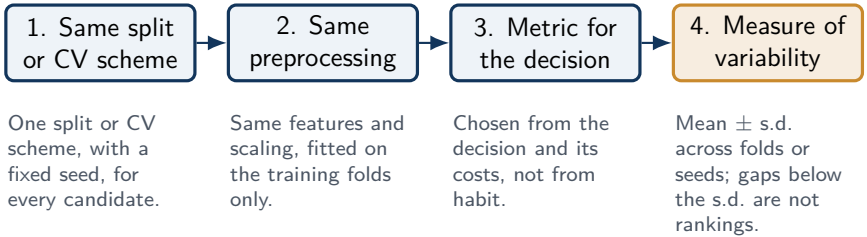
What to track, sources of bias, hyperparameter search, Weights & Biases

1. Modeling | 2. Tracking | 3. Registry | 4. Evaluation | 5. Decisions | 6. Practice

What must be tracked, and why

Dataset version or extraction date	Allows a result to be reproduced on the same data.
Feature pipeline version	Separates the effect of preprocessing changes from that of model changes.
Hyperparameters and random seeds	Identify the exact configuration that produced a result.
Metrics, confusion matrices, artifacts	Allow the trained model, not only a number, to be audited.
Code commit or notebook snapshot	Allows the logic behind a run to be replayed later (Class 17).

A fair comparison: hold everything else fixed



Why it matters

A difference between two models is evidence only if the data, the preprocessing (unless preprocessing is the experiment), and the metric are identical, and if the difference exceeds the run-to-run variability.

Sources of bias and their solutions

Source of bias	Solution
Test-set reuse. Reusing the test set until it becomes part of tuning (model-selection leakage).	Tune on a validation split or with cross-validation; use the test set once, at the end.
Hidden state. Hidden notebook state and untracked edits.	Log commit, configuration, seed, and data version with every run; restart the kernel and run all cells.
Selective reporting. Reporting only the best run.	Report all runs and the search budget; give mean $\pm$ s.d. over folds or seeds.

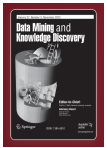
Hyperparameter search strategies

Manual tuning	Small problems; useful to build intuition.
Grid search	Exhaustive evaluation of a small, structured search space.
Random search	Better coverage when only a few of many hyperparameters matter (Bergstra and Bengio, 2012).

Optimization-based search: the validation score is treated as a black-box function.

Bayesian optimization	A surrogate model of the score (Gaussian process, TPE) guides the next trial; suited to expensive models.
Genetic algorithms	A population of configurations evolves through selection, crossover, and mutation.
Other heuristics	Simulated annealing, particle swarm, and bandit-based early stopping (successive halving, Hyperband).

Case study: how much does tuning help a decision tree?



Mantovani, Horváth, Rossi, Cerri, Barbon Junior, Vanschoren, de Carvalho (2024). *Better trees: an empirical study on hyperparameter tuning of classification decision tree induction algorithms*. Data Mining and Knowledge Discovery, 38(3), 1364–1416.

Experimental design

- **Models and data:** CART and C4.5; 94 OpenML classification datasets.
- **Six techniques:** random search, genetic algorithm, particle swarm, estimation of distribution, model-based optimization, Irace.
- **Protocol:** nested CV (10 outer, 3 inner folds); same partitions and seeds; 900 evaluations.
- **Baseline:** default hyperparameters.

Findings

- Significant gains over the defaults: **CART** in about two thirds of the datasets, **C4.5** in one third.
- **Irace** was the best technique; few evaluations were enough.
- **A small subset of hyperparameters** explains most of the gain (CART: `minsplit`, `minbucket`).
- **Defaults can already be good**.

References and materials

Papers and materials

- Hastie, Tibshirani, and Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009: model classes, regularization, and evaluation.
- Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006: probabilistic foundations of supervised learning.
- Pedregosa et al. "Scikit-learn: Machine Learning in Python." *JMLR*, 2011; and the scikit-learn user guide: estimator APIs, preprocessing, and pipelines.
- Mantovani et al. "Better trees: an empirical study on hyperparameter tuning of classification decision tree induction algorithms." *Data Mining and Knowledge Discovery*, 2024: the case study discussed here.

Take-away: modeling is a disciplined choice

Choose a model family because its assumptions fit the problem; train it through a single reproducible pipeline; and compare alternatives with evaluation evidence rather than intuition.

- **The estimator interface** separates fitting, transformation, and prediction while making components composable.
- **Pipelines** prevent training–serving and preprocessing mismatches by treating data preparation as part of the model.
- **The next question** is how to retain the evidence for every run, artifact, and decision.

Next: Class 19 — ML experiment tracking, registry, evaluation, and decision-making.